agentic PBT × 真实 CVE:全流程与对应代码

将每个项目回退至补丁提交之前,使「agent 是否发现补丁所修复的缺陷」成为可判定的问题

阶段一 · 数据集构建

1 获取漏洞公告
保留附 CVE 编号与修复提交的记录
`dataset/build_dataset.py`

↓ 2942 行 / 694 个包

2 定位受影响函数
解析补丁提交之前的源码
`dataset/enrich_functions.py`

↓ vuln_functions 列

3 排除平台相关样本
标记仅在 Windows 成立的缺陷
`dataset/platform_scan.py`

↓ 风险样本 17%

4 回退至补丁前版本
检出修复提交的父提交
`dataset/materialize.py`

17 个样本

补丁前的源码 + 指定的目标模块
+ 已知的标准答案

阶段二 · 实验运行

5 发起单次运行
串行调度,配额超限后自动重试
`runs/serial.sh · go.sh`

↓ `claude -p "/hypo_v2 <模块>"`

6 agent 按提示词执行 8 步 `hypo_v2.md`

1. 确定测试目标
2. 阅读实现
3. 记录代码声明的规约 → claims.md ★
4. 依据 claims.md 推导性质 ★
5. 编写测试,逐条标注所依据的规约 ★
6. 运行测试,分类失败原因
7. 撰写报告
8. 给出结论

↓ 单次运行的产物

`claims.md`	代码声明的规约
`test_*.py`	性质测试
`bug_report_*.md`	缺陷报告与复现脚本
`claude_result.json`	运行回合数与开销

阶段三 · 结果度量

7 还原全部提出的性质
不限于失败的,通过的亦计入
`dataset/extract_properties.py`

↓ 每条性质一行

8 判定是否命中
三个相互独立的信号
`dataset/classify_hit.py`

↓ 每次运行一行

9 5 次运行归并为一档
稳定命中 / 不稳定 / 从未命中
`dataset/aggregate.py`

↓ 三档分类

10 事前可计算的筛选量
无需运行,用于样本预筛
`dataset/check_i3.py`

最终产出

`table1_cve.csv` `table2_results.csv`
`table3_stability.csv`

方框右下角 = 执行该步骤的代码文件　　★ = 为使规约可追溯而新增的步骤